COS214 Practical 6

✨TIVIDY: Workflow Management System ✨

-Jayden Smith - u24583741
-Declan Mc Cullough -u21760022
-Yash Ghela -u25161360
-Rei Piater-Boswell -u25678592
-Leanne van der Horst - u24566935

Task1: Research Workflow Management Systems

When designing our workflow management system we found that a workflow is defined as a system for managing repetitive processes and tasks which occur in a particular order, a workflow management system defines, manages and executes workflows. A workflow definition is static while a running workflow is the actual running instance of the workflow. In most systems, work items and activities are separated as static definitions vs active processes, this allows for them to be managed differently. When assigning items we found it was best to use a role based map that allows for an item to be assigned to the relevant participant as fast as possible, if they cannot do it then that item needs to be moved to the closest next person who can handle the item. Generally, participants are registered with personal details, a role such as intern, manager etc and then a system defined responsibility level that can be used when mapping roles. We have built our approval, rejection and escalation system around a reviewer role that will be given to a participant rather than an automated system as our scenario will call for a more detailed look from a person rather than system. We have modelled our system such that certain items will depend on other work items to be completed before they can begin, allowing for a sequential and parallel workflow system. When events occur the relevant states are changed to pass data when a message is received, if that event fails we use versioning to rollback to the last stable state and preserve data. Pre built APIs or connectors are generally the best way to interact with external systems. In order to maintain a history, snapshots of a system are taken either in real time or in a pre defined gap. This creates a responsive roll back system. Running workflows are monitored using analytics systems that allow you to see who is working where and on what tasks, as well as computing the amount of changes occurring. A workflow engine is reusable across multiple different processes, because it generalises how work is done, not what work is done, a restaurant can use a workflow system and a movie studio can use the same system if their workflows require the same elements. It decouples the process from a blueprint. An important aspect of a system is to make nested activities directly linked to their parent so they can easily move and change states within while acting as dependencies. For workflow systems to work for different processes, being able to configure internal business rules is important. When work is in progress but we need to change workflow definitions, we have to send notifications out to the relevant work items including cancellations, delays or changes where relevant. All these details allow for a powerful workflow system that can handle and easily adjust to business needs.

References:

https://www.ibm.com/think/topics/workflow
https://www.sciencedirect.com/topics/computer-science/workflow-management-system
https://www.ibm.com/docs/en/openpages/9.0.0?topic=fundamentals-workflow-definitions-workflow-instances
https://www.atlassian.com/work-management/project-management/approval-process-workflow
https://www.ibm.com/think/topics/workflow-engine

Task2: Define the Scenario and System

Initial scope of TIVIDY

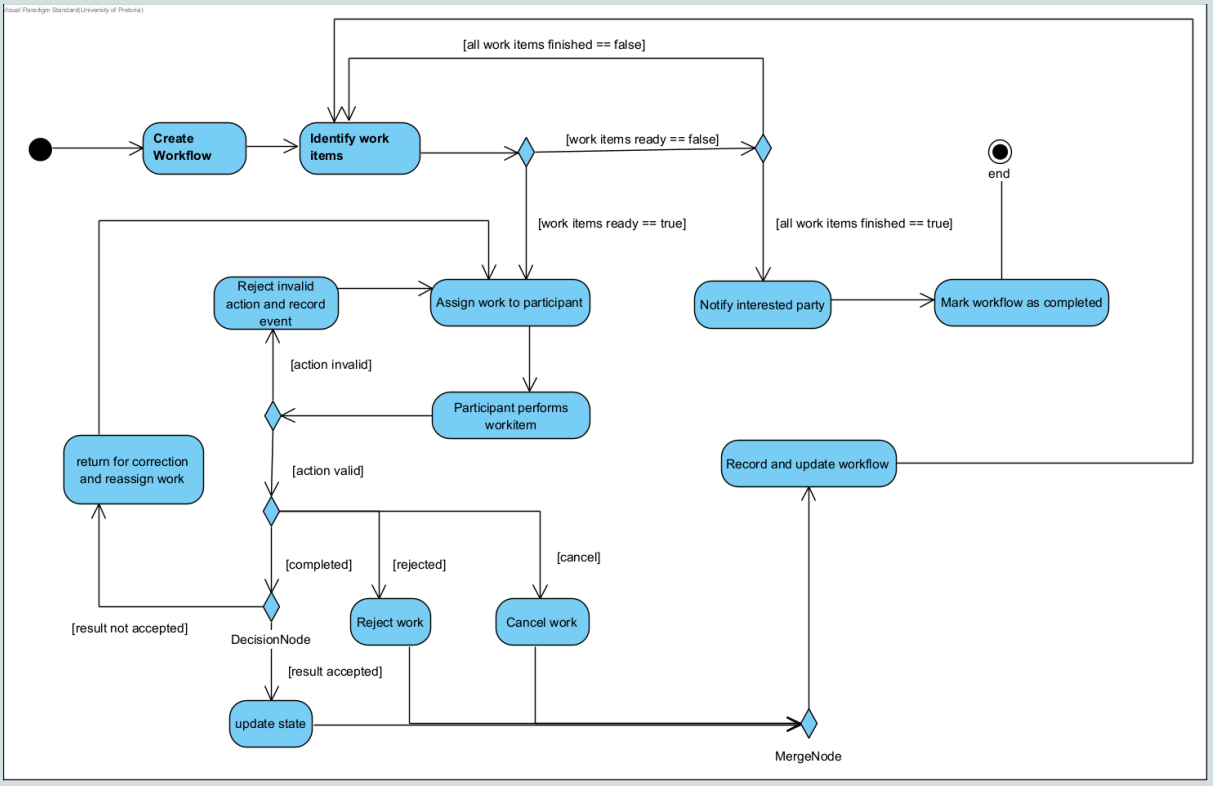
TIVIDY is a generic workflow management system that defines, manages and executes workflows. It generalises how work is done rather than what work is done, so the same engine could serve a restaurant or an animation studio. Its main responsibilities are creating running instances from static definitions, managing work-item lifecycles and dependencies. It also reacts to events, integrates with external systems through connectors, keeps snapshots and an audit history for rollback, and provides monitoring and analytics. A definition holds the static structure, dependencies, roles and rules, while each instance holds its own state. TIVIDY coordinates work but does not perform it, and we expect assignment rules, approval structure, work-item behaviour, integrations and reporting to vary independently.

- **Organization and environment:** Our animation studio is a mid-sized studio producing animated shorts for external clients. About 15 people work across departments, and many tasks cannot start until others finish. Handovers and approvals are currently tracked through chat and spreadsheets, which causes lost approvals and unclear versions.
- **Process being managed:** The production of the short from client brief to final delivery. The main workflow contains one nested sub-process for each shot.
- **Participants and roles:** Interns and junior artists handle simple tasks, and artists such as animators and compositors do the core production work. Pipeline TDs handle technical failures, the Art Director and Director act as reviewers, and the Producer handles escalations and scope changes. The client is an external reviewer at defined milestones.
- **Major pieces of work:** Pre-production covers the script, storyboard, animatic and designs. Each shot then goes through layout, animation, rendering and compositing, followed by post-production (sound, edit, colour grade) and delivery.
- **Decisions and routing points:** Each task goes to the participant with the matching role and lowest current load, and moves to the next higher responsibility level if they cannot take it. Every review ends in approved, revisions needed or rejected, with escalation to the Producer after a set number of revision rounds.
- **Approvals and dependencies:** The animatic needs Director and then Client approval before production begins, and character designs need Art Director approval before modelling. Animation needs an approved layout, and compositing needs all render passes. Shots can run in parallel, but the final edit needs every shot composited.
- **Failures and exceptions:** A reviewer who does not respond in time causes the review to escalate. A failed render is retried and then escalated to the Pipeline TD, and a failed event or client rejection rolls the workflow back to its last stable or approved state. If a shot is cut, its work items are cancelled and dependants are notified.
- **External systems:** A render farm receives jobs and returns pass or fail results. Asset management stores files and versions, a client review portal handles client approvals, and a notification service sends emails or chat messages. An invoicing system is used at delivery.

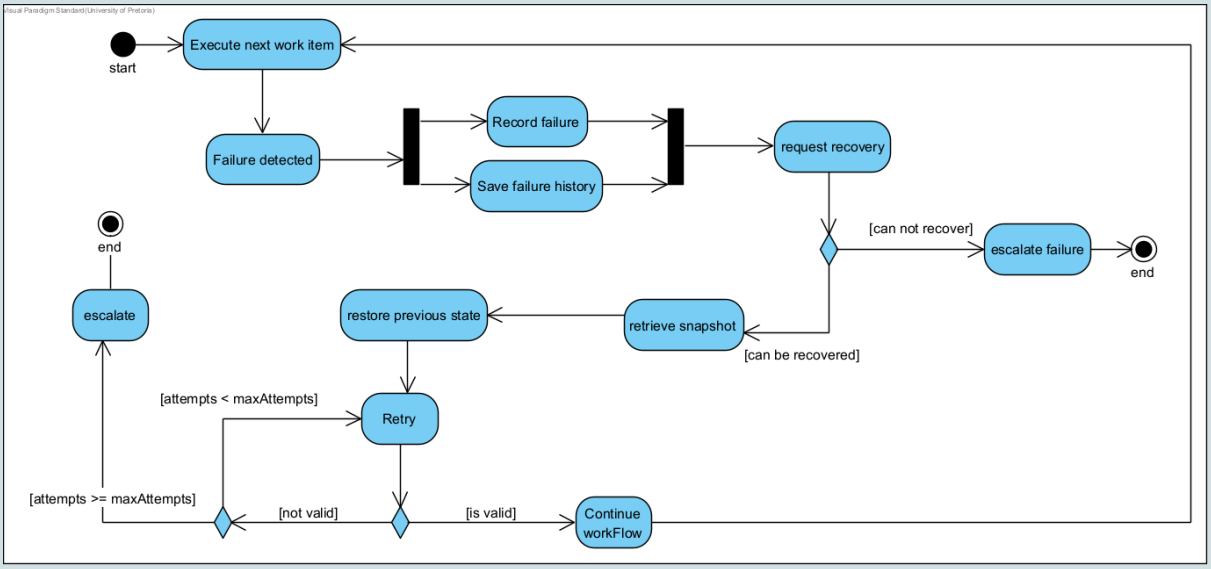
- **Expected outcome:** The approved short is delivered to the client. Every assignment, approval, revision, failure and rollback is recorded, and reports show who worked on what and where time was lost.

Task3: UML activity diagrams

Activity diagram 1 - general workflow execution

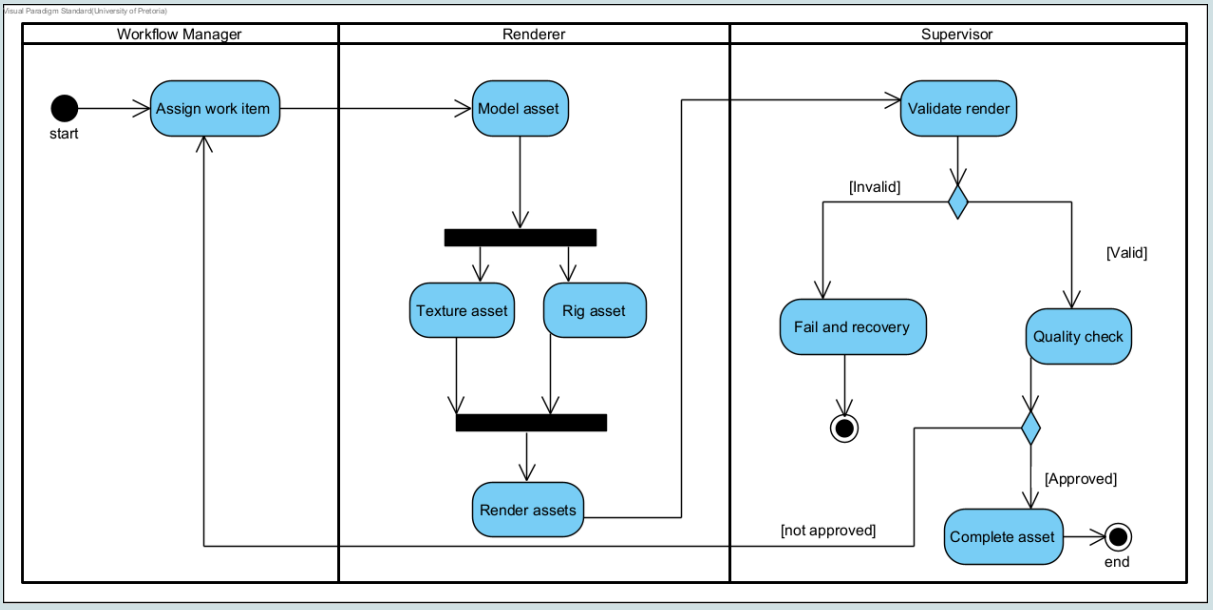


Activity diagram 2 - general workflow execution. Failure and recovery.



Activity diagram 3 - scenario diagram

Workflow of manager assigning work item to Renderer and work item being executed and validated by supervisor



Task4:

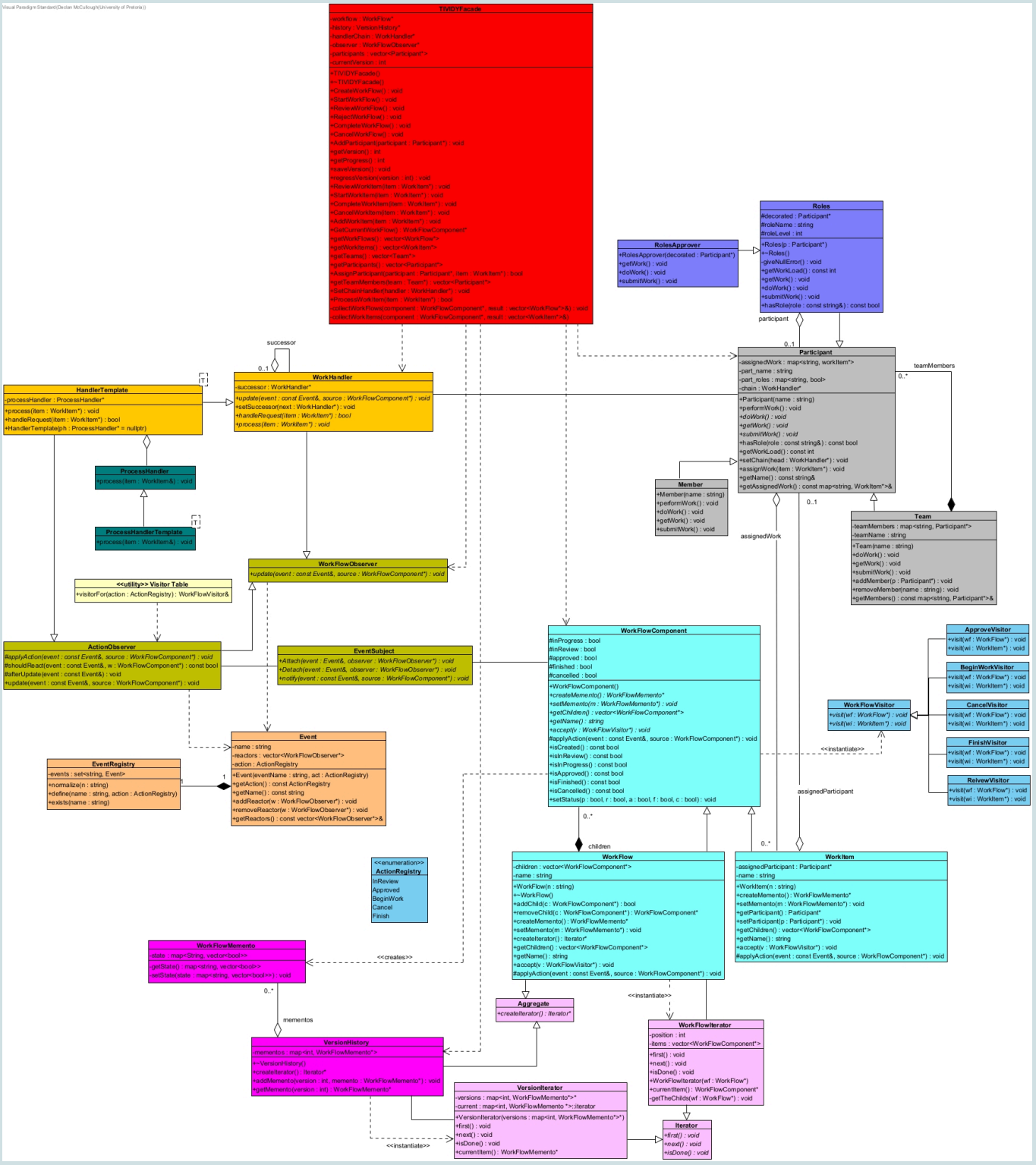
Pattern	Use
Composite Participants 1: ● Component : ○ WorkFlowComponent ● Composite : ○ WorkFlow ● Leaf : ○ WorkItem ● Client : ○ TIVIDYFacade Participants 2: ● Component : ○ Participant ● Composite : ○ Team ● Leaf : ○ Employee ● Client : ○ TIVIDYFacade	Composite 1 ● solves the need to treat multiple work items uniformly and have them be in a part-whole hierarchy within work flows. ● WorkFlowComponent provides an interface for TIVIDYFacade; WorkFlow contains WorkItems and potentially other WorkFlows. ● The part that can potentially change are the functions within the pattern and the use of WorkFlow containing other WorkFlow objects. Composite 2 ● solves the need to treat multiple employees uniformly and have them be in a part-whole hierarchy within teams. ● Participant provides an interface for TIVIDYFacade; Team contains Employees and potentially other Teams. ● The part that can potentially change are the functions within the pattern and the use of Teams containing other Team objects.
Decorator ○ Participants 2: ● Component: ○ Participant ● ConcreteComponent: ○ Employee ● Decorator : ○ RoleDecorator ● ConcreteDecorators: ○ ApproverRole	Decorator 2 ● The Decorator design pattern allows the attachment of additional responsibilities to Employees at run time. This will allow for the use of a generic Employee that can be added onto to make it more unique and adapted to specified roles in the company and animation pipeline. ● The Participant will act as an interface for Employees that can have ConcreteDecorators added to them; The RoleDecorator will hold a reference to an Employee object. The ConcreteDecorators will then add stackable responsibilities to the Employee object such as being able to approve or disapprove workflows and work items. ● The part that can potentially change are the functions within the pattern as well as the actual types of ConcreteDecorators.
Facade Participants ● Facade : ○ TIVIDYFacade	● TIVIDYFacade will allow a user to interact with the whole system at a simplified level so they do not have to know all the ins and outs of every pattern. It helps connect patterns to each other

• SubSystems : ○ WorkFlow ○ Participant ○ Connector ○ VersionHistory	too. • Within the facade the user will have the ability to begin, review, reject, cancel or complete a workflow; add a participant to a workItem; get progress of a workflow; get, save or regress a version of a workItem • The parts that can potentially change are the functions within the pattern and what it can do since some parts a manager should only be able to do.
Chain of Responsibility Participants • Handler : ○ WorkHandler • Client : ○ Participant • ConcreteHandlers : ○ Handler Template	• Gives multiple participants the chance to handle the workItem that is in progress, while decoupling the decorators of the participants. Chains the employee objects and sends the workItem to the next employee with that role if they have less of a workload. • WorkHandler defines the interface for handling the requests and implements the chaining process by linking the next employee to the chain; Participant will initiate the request to be sent down the chain when a workItem enters the State corresponding to a ConcreteHandler; The ConcreteHandlers will handle the request of the WorkItem object being passed along the chain until it reaches an employee with a lower workload. • The parts that can potentially change are the functions within the pattern and what ConcreteHandlers we have since the system may have to change. • Concrete Handlers use a template class so that different scenarios can create their own handlers where necessary
Iterator Participants • Iterator : ○ Iterator • ConcreteIterator : ○ VersionIterator ○ WorkFlowIterator • Aggregate : ○ Aggregate • ConcreteAggregate : ○ WorkFlow ○ VersionHistory	• The Iterator will be able to go through the composite of work items to check the number of WorkItems as well as iterate through the memento map to count version numbers, all to get statistics of WorkFlows. • The Iterator defines the interface for accessing and traversing through the WorkFlow and the VersionHistory; the ConcreteIterators implement the Iterator and go through these ConcreteAggregates by using the Aggregate as an interface. • We believe there will be little to no changes in this pattern, except adding a new ConcreteIterator to traverse other patterns like counting the number of current employees.
Memento Participants • Originator :	• The Memento will be able to hold revisions of workItems and WorkFlows and will allow restoration and tracking of their states.

○ WorkFlowComponent ● Memento : ○ WorkFlowMemento ● CareTaker : ○ VersionHistory	● The WorkFlowComponent holds a state at all times, and creates a snapshot of this state and this gets stored in the WorkFlowMemento and the VersionHistory keeps the WorkFlowMemento safe. ● The use of what states are stored in the snapshot might change.
Observer Participants ● Subject : ○ EventSubject ● ConcreteSubject: ○ WorkFlow ○ WorkItem ● Observer : ○ WorkFlowObserver ○ ActionObserver ● ConcreteObserver : ○ WorkItem ○ WorkHandler	● This Pattern solves the problem of multiple other patterns having to know and change according to the WorkFlowComponents automatically. The Adapter, the WorkItems, and the Handlers. ● Events are defined by the user and have a name, an action and a set of reactors. Reactors are the classes that must react to an event when it is fired, actions are a predefined set of actions that an update can cause, these generally just change the visitor state of a class. All events are stored in an event registry. ● The EventSubject defines an interface for observers to attach and detach to the WorkFlows and the WorkItems. The WorkFlow observer defines the update function which the ActionObserver then defines into a template method algorithm with swappable action steps that react to an event at runtime. ● The number of classes that observe the WorkFlows and WorkItems may need to change and the Subjects may need to include other object types
Visitor Participants ● Visitor : ○ WorkFlowVisitor ● ConcreteStates : ○ FinishVisitor ○ BeginWorkVisitor ○ ReviewVisitor ○ FinishVisitor ○ CancelVisitor ● Element : ○ WorkFlowComponent ● ConcreteElements: ○ WorkFlow ○ WorkItem ● Client : ○ TIVIDYFacade	● The Visitor allows for double dispatch so that WorkItems and WorkStates can be handled differently when changing their boolean values, but allows the states of the different classes to stay identical as they are still both WorkFlowComponents. ● The WorkFlowVisitor defines the interface for double dispatch behaviour. The ConcreteVisitors do the actual visiting
Template Method	● The Template method allows one function to be

Participants ● AbstractClass : ○ Participant ● ConcreteClass : ○ Employee ○ Team ● templateMethod : ○ performWork ● primitiveOperation : ○ doWork ○ getWork ○ submitWork Action Observer ● Abstract Class: ActionObserver ● ConcreteClass: WorkItem ● templateMethod: applyAction	called in the Participant Composite and depending on the decorator, the primitive operations do different actions. ● The Participant implements performWork as the template method and defines abstract primitive operations, doWork, getWork, submitWork, which then get implemented in the ConcreteClasses. ● We do not foresee a change in the Template Method unless it gets changed for a different design pattern. Action Observer uses a template method that is explained in the Observer section.
Strategy ProcessHandler ● Context: HandlerTemplate ● Strategy: ProcessHandler ● Concrete Strategy: ProcessHandlerTemplate	Different levels of the chain of responsibility can handle their work differently by using the strategy class. This uses a template class so user defined strategies can be added

Task5:

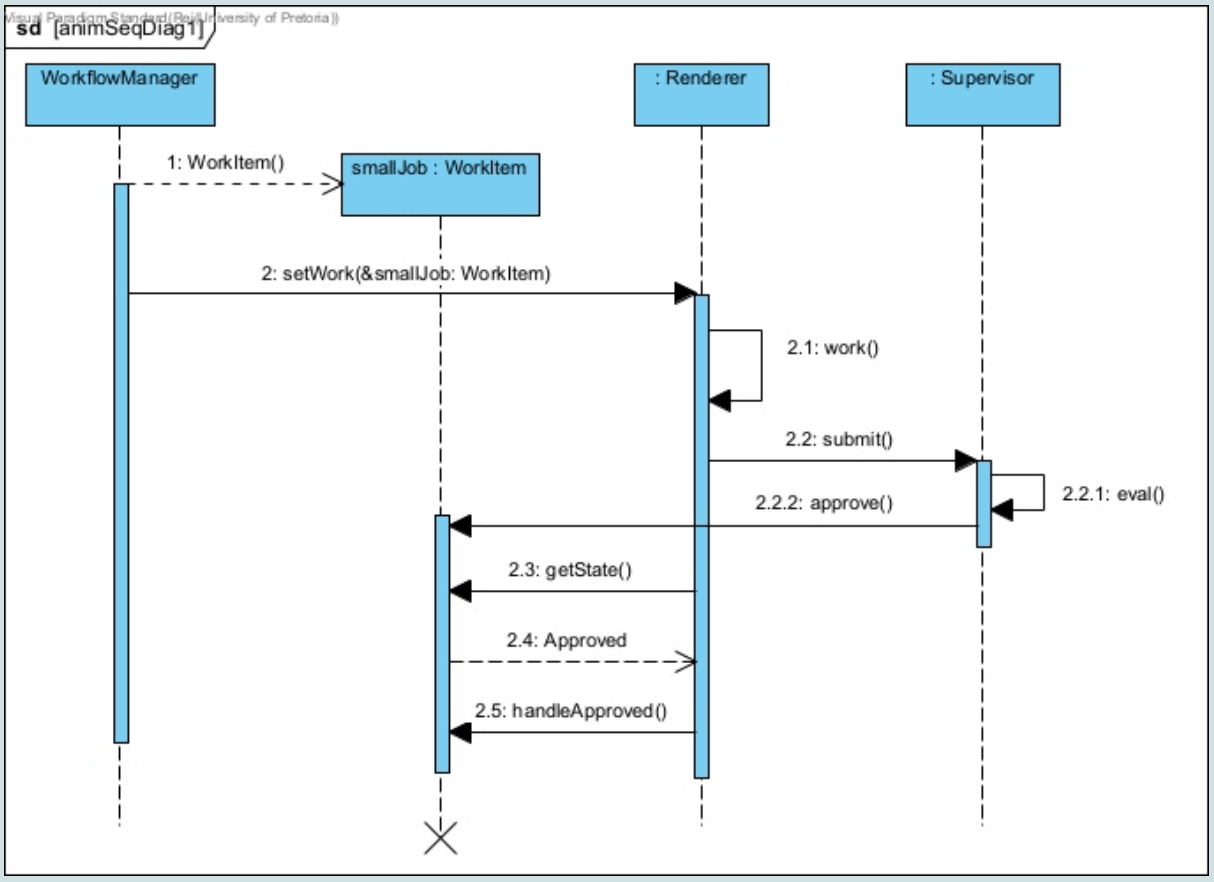

Visual Paradigm Standard(Declan McCullough(University of Pretoria))
TiVIDYFacade
-workflow : WorkFlow*
-history : VersionHistory*
-handlerChain : WorkHandler*
-observer : WorkFlowObserver*
-participants : vector<Participant*>
-currentVersion : int
+TiVIDYFacade()
+~TiVIDYFacade()
+CreateWorkFlow() : void
+StartWorkFlow() : void
+ReviewWorkFlow() : void
+RejectWorkFlow() : void
+CompleteWorkFlow() : void
+CancelWorkFlow() : void
+AddParticipant(participant : Participant*) : void
+getVersion() : int
+getProgress() : int
+saveVersion() : void
+regressVersion(version : int) : void
+ReviewWorkItem(item : WorkItem*) : void
+StartWorkItem(item : WorkItem*) : void
+CompleteWorkItem(item : WorkItem*) : void
+CancelWorkItem(item : WorkItem*) : void
+AddWorkItem(item : WorkItem*) : void
+GetCurrentWorkFlow() : WorkFlowComponent*
+getWorkFlows() : vector<WorkFlow*>
+getWorkItems() : vector<WorkItem*>
+getTeams() : vector<Team*>
+getParticipants() : vector<Participant*>
+AssignParticipant(participant : Participant*, item : WorkItem*) : bool
+getTeamMembers(team : Team*) : vector<Participant*>
+SetChainHandler(handler : WorkHandler*) : void
+ProcessWorkItem(item : WorkItem*) : bool
-collectWorkFlows(component : WorkFlowComponent*, result : vector<WorkFlow*>&) : void
-collectWorkItems(component : WorkFlowComponent*, result : vector<WorkItem*>&)
RolesApprover
+RolesApprover(decorated : Participant*)
+getWork() : void
+doWork() : void
+submitWork() : void
Roles
#decorated : Participant*
#roleName : string
#roleLevel : int
+Roles(p : Participant*)
+~Roles()
-giveNullError() : void
+getWorkLoad() : const int
+getWork() : void
+doWork() : void
+submitWork() : void
+hasRole(role : const string&) : const bool
participant
0..1
successor
0..1
HandlerTemplate
-processHandler : ProcessHandler*
+process(item : WorkItem*) : void
+handleRequest(item : WorkItem*) : bool
+HandlerTemplate(ph : ProcessHandler* = nullptr)
WorkHandler
-successor : WorkHandler*
+update(event : const Event&, source : WorkFlowComponent*) : void
+setSuccessor(next : WorkHandler*) : void
+handleRequest(item : WorkItem*) : bool
+process(item : WorkItem*) : void
Participant
-assignedWork : map<string, workItem*>
-part_name : string
-part_roles : map<string, bool>
-chain : WorkHandler*
+Participant(name : string)
+performWork() : void
+doWork() : void
+getWork() : void
+submitWork() : void
+hasRole(role : const string&) : const bool
+getWorkLoad() : const int
+setChain(head : WorkHandler*) : void
+assignWork(item : WorkItem*) : void
+getName() : const string&
+getAssignedWork() : const map<string, WorkItem*>&
teamMembers
0..*
ProcessHandler
+process(item : WorkItem&) : void
ProcessHandlerTemplate
+process(item : WorkItem&) : void
Member
+Member(name : string)
+performWork() : void
+doWork() : void
+getWork() : void
+submitWork() : void
0..1
assignedWork
Team
-teamMembers : map<string, Participant*>
-teamName : string
+Team(name : string)
+doWork() : void
+getWork() : void
+submitWork() : void
+addMember(p : Participant*) : void
+removeMember(name : string) : void
+getMembers() : const map<string, Participant*>&
WorkFlowObserver
+update(event : const Event&, source : WorkFlowComponent*) : void
<<utility>> Visitor Table
+visitorFor(action : ActionRegistry) : WorkFlowVisitor&
ActionObserver
#applyAction(event : const Event&, source : WorkFlowComponent*) : void
#shouldReact(event : const Event&, w : WorkFlowComponent*) : const bool
#afterUpdate(event : const Event&) : void
+update(event : const Event&, source : WorkFlowComponent*) : void
EventSubject
+Attach(event : Event&, observer : WorkFlowObserver*) : void
+Detach(event : Event&, observer : WorkFlowObserver*) : void
+notify(event : const Event&, source : WorkFlowComponent*) : void
WorkFlowComponent
#inProgress : bool
#inReview : bool
#approved : bool
#finished : bool
#cancelled : bool
+WorkFlowComponent()
+createMemento() : WorkFlowMemento*
+setMemento(m : WorkFlowMemento*) : void
+getChildren() : vector<WorkFlowComponent*>
+getName() : string
+accept(v : WorkFlowVisitor*) : void
#applyAction(event : const Event&, source : WorkFlowComponent*) : void
+isCreated() : const bool
+isInReview() : const bool
+isInProgress() : const bool
+isApproved() : const bool
+isFinished() : const bool
+isCancelled() : const bool
+setStatus(p : bool, r : bool, a : bool, f : bool, c : bool) : void
ApproveVisitor
+visit(wf : WorkFlow*) : void
+visit(wi : WorkItem*) : void
BeginWorkVisitor
+visit(wf : WorkFlow*) : void
+visit(wi : WorkItem*) : void
WorkFlowVisitor
+visit(wf : WorkFlow*) : void
+visit(wi : WorkItem*) : void
CancelVisitor
+visit(wf : WorkFlow*) : void
+visit(wi : WorkItem*) : void
FinishVisitor
+visit(wf : WorkFlow*) : void
+visit(wi : WorkItem*) : void
ReivewVisitor
+visit(wf : WorkFlow*) : void
+visit(wi : WorkItem*) : void
<<instantiate>>
assignedParticipant
Event
-name : string
-reactors : vector<WorkFlowObserver*>
-action : ActionRegistry
+Event(eventName : string, act : ActionRegistry)
+getAction() : const ActionRegistry
+getName() : const string
+addReactor(w : WorkFlowObserver*) : void
+removeReactor(w : WorkFlowObserver*) : void
+getReactors() : const vector<WorkFlowObserver*>&
EventRegistry
-events : set<string, Event>
+normalize(n : string)
+define(name : string, action : ActionRegistry)
+exists(name : string)
1
0..*
children
<<enumeration>>
ActionRegistry
InReview
Approved
BeginWork
Cancel
Finish
WorkFlow
-children : vector<WorkFlowComponent*>
-name : string
+WorkFlow(n : string)
+~WorkFlow()
+addChild(c : WorkFlowComponent*) : bool
+removeChild(c : WorkFlowComponent*) : WorkFlowComponent*
+createMemento() : WorkFlowMemento*
+setMemento(m : WorkFlowMemento*) : void
+createIterator() : Iterator*
+getChildren() : vector<WorkFlowComponent*>
+getName() : string
+accept(v : WorkFlowVisitor*) : void
#applyAction(event : const Event&, source : WorkFlowComponent*) : void
WorkItem
-assignedParticipant : Participant*
-name : string
+WorkItem(n : string)
+createMemento() : WorkFlowMemento*
+setMemento(m : WorkFlowMemento*) : void
+getParticipant() : Participant*
+setParticipant(p : Participant*) : void
+getChildren() : vector<WorkFlowComponent*>
+getName() : string
+accept(v : WorkFlowVisitor*) : void
#applyAction(event : const Event&, source : WorkFlowComponent*) : void
WorkFlowMemento
-state : map<String, vector<bool>>
-getState() : map<string, vector<bool>>
-setState(state : map<string, vector<bool>>) : void
<<creates>>
0..*
mementos
Aggregate
+createIterator() : Iterator*
<<instantiate>>
WorkFlowIterator
-position : int
-items : vector<WorkFlowComponent*>
+first() : void
+next() : void
+isDone() : void
+WorkFlowIterator(wf : WorkFlow*)
+currentItem() : WorkFlowComponent*
-getTheChilds(wf : WorkFlow*) : void
VersionHistory
-mementos : map<int, WorkFlowMemento*>
+~VersionHistory()
+createIterator() : Iterator*
+addMemento(version : int, memento : WorkFlowMemento*) : void
+getMemento(version : int) : WorkFlowMemento*
VersionIterator
-versions : map<int, WorkFlowMemento*>*
-current : map<int, WorkFlowMemento *>::iterator
+VersionIterator(versions : map<int, WorkFlowMemento*>*)
+first() : void
+next() : void
+isDone() : void
+currentItem() : WorkFlowMemento*
<<instantiate>>
Iterator
+first() : void
+next() : void
+isDone() : void

Task6:

OLD DIAGRAMS:

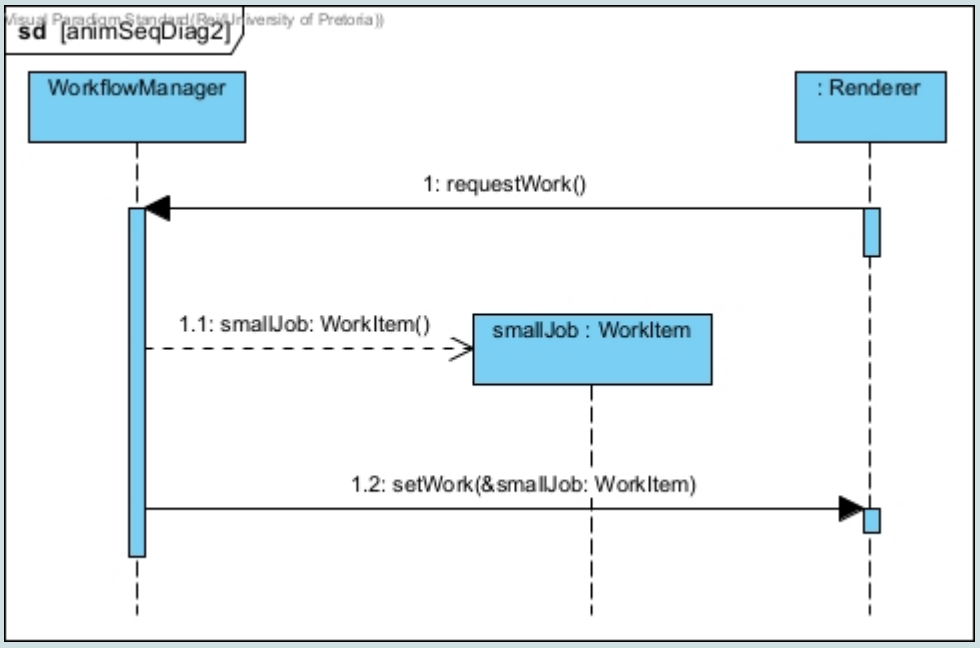
Sequence Diagram 1:

Creating a job, working on it and handle what reviewer “says”



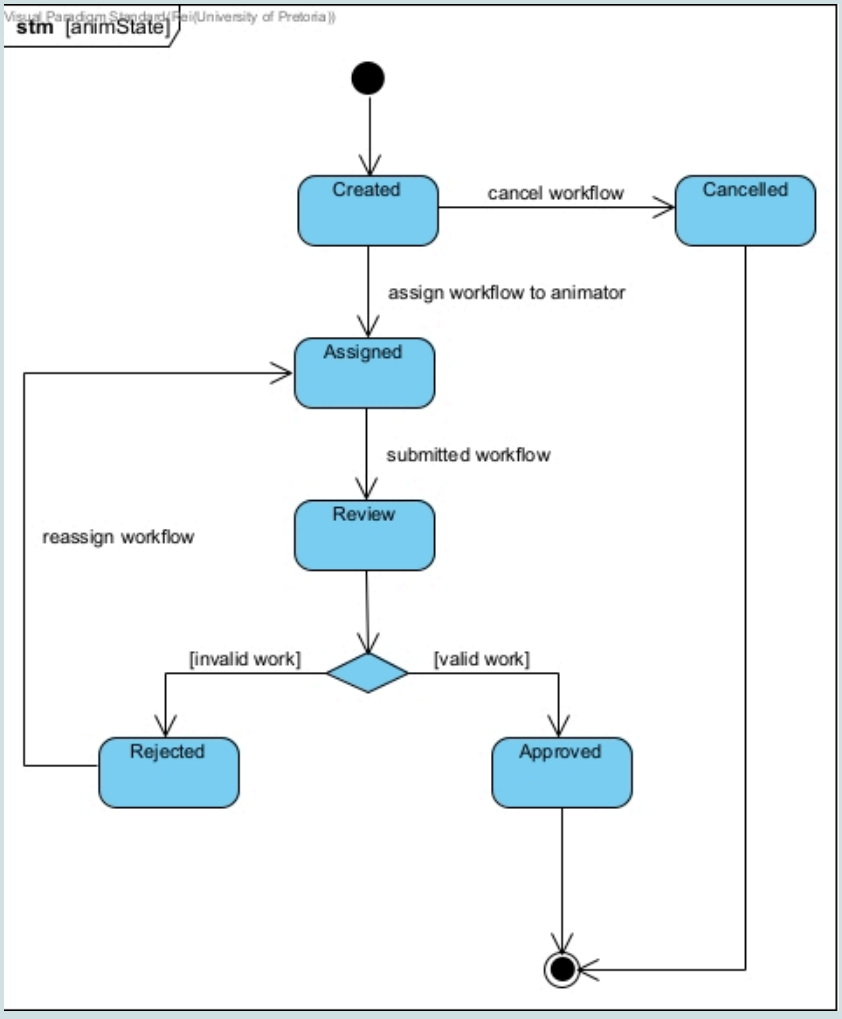
Sequence Diagram 2:

Handling an event where a worker requests to have a WorkItem



State Diagram:

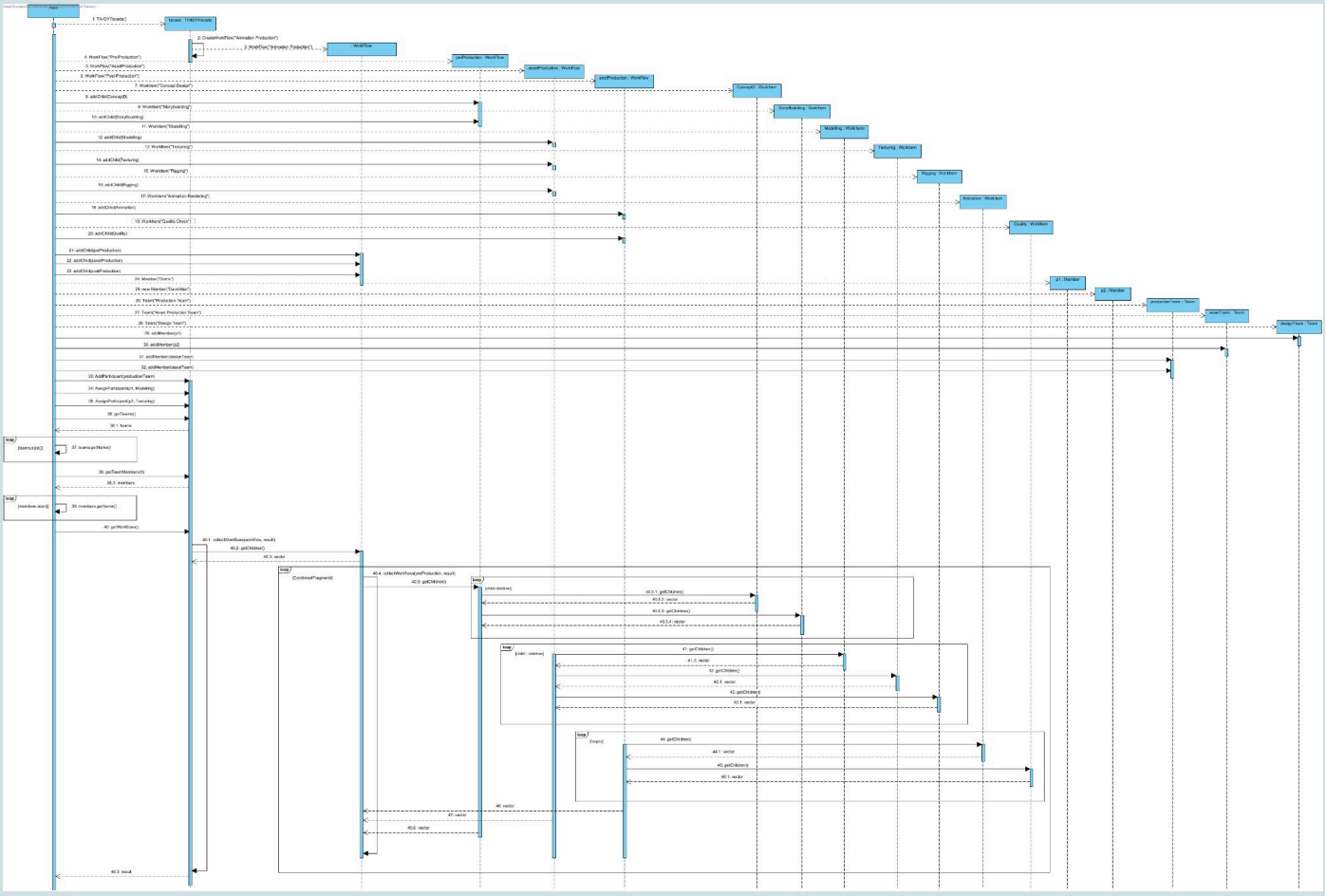
State of WorkItem



NEW DIAGRAMS:

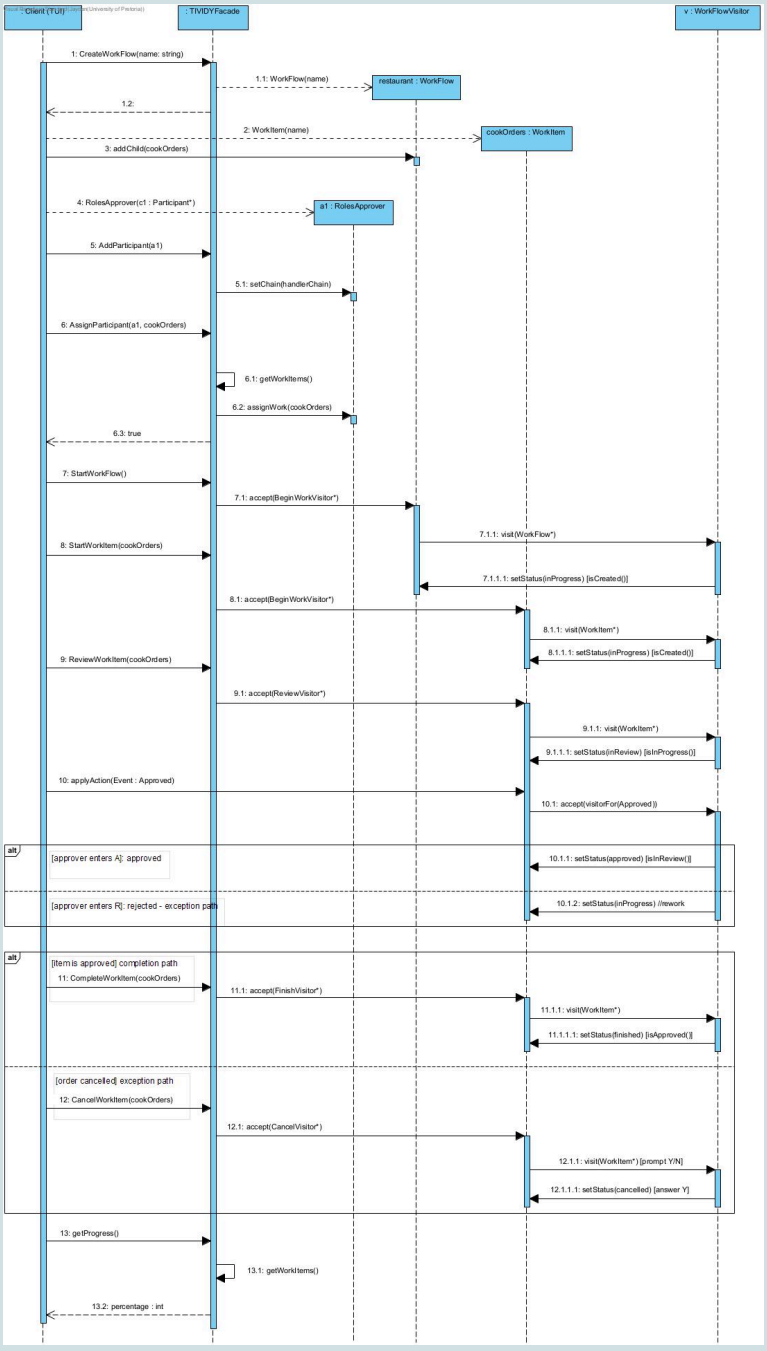
Sequence Diagram 1:

Animation Studio Scenario:



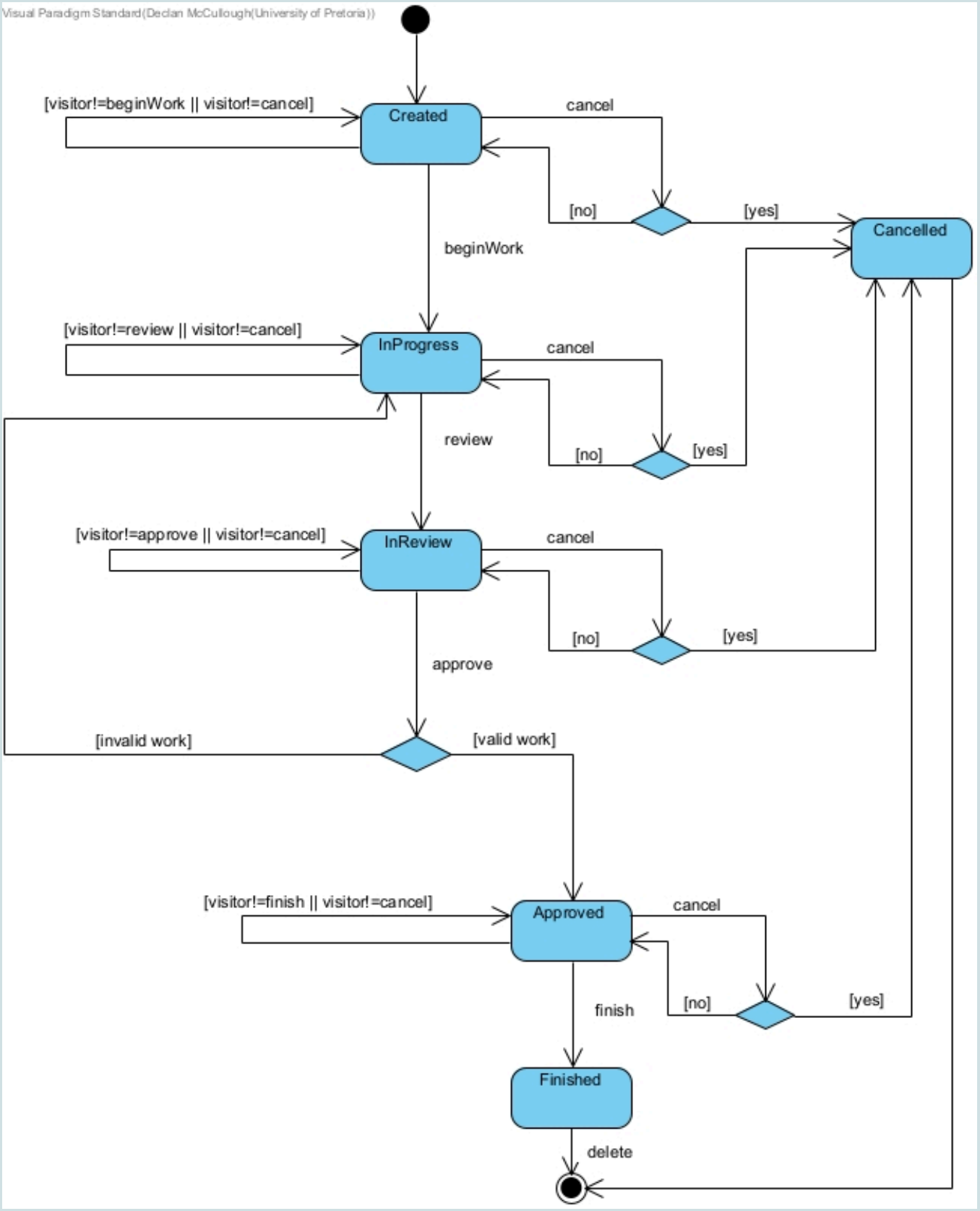
Sequence Diagram 2:

Restaurant Scenario:



State Diagram:

State of WorkFlowComponents:


Visual Paradigm Standard(Declan McCullough(University of Pretoria))
[visitor!=beginWork || visitor!=cancel]
Created
cancel
[no]
[yes]
Cancelled
beginWork
[visitor!=review || visitor!=cancel]
InProgress
cancel
review
[no]
[yes]
[visitor!=approve || visitor!=cancel]
InReview
cancel
[no]
[yes]
approve
[invalid work]
[valid work]
[visitor!=finish || visitor!=cancel]
Approved
cancel
finish
[no]
[yes]
Finished
delete

Task7:

2026/10/04

- **Yash:** Added research for Prac 6 task 1, including citing references.

2026/10/21

- **Yash:** Uploaded an updated UML class diagram that uses templates and strategy, for the team to review.

2026/10/25

- **Declan:** Updated the Composite to use the State pattern for all calls that relate to the state it holds.
- **Yash:** Built the Participant composite, and the Chain of Responsibility and Strategy with their templates. Updated the VPP file. Work items can't be connected until more of the system is built.
- **Declan:** Added Double Dispatch to WorkItemState and the WorkFlow Composite, so it also acts as a Visitor pattern.

2026/10/26

- **Rey:** Added the Decorator `.h` and `.cpp` files to `Project/include` and `Project/src`.
- **Rey:** Added a `Dockerfile` and `compose.yml` to the root directory.
- **Rey:** Changed the main VPP file to reflect the Decorator pattern.
- **Rey:** Added the Sequence and State diagrams' VPP file to `Diagrams/VPP files`, and their exported images to `Diagrams/Exported_Diagrams`.

2026/10/31

- **Declan:** `setMemento()` and `createMemento()` now call `getState()` and `setState()` from the Memento pattern. The map uses `std::string` as the key and `vector<bool>` as the value, replacing `WorkItemState`.
- **Rey:** Changed the Dockerfile to copy all files into the work directory so the Makefile works.
- **Rey:** Updated `README.md` with commands to run the project through Docker Compose, and to use Valgrind and GDB in Docker Compose.
- **Rey:** Changed `compose.yml` to create containers for the main program, Valgrind and GDB.
- **Rey:** Changed the Roles Decorator to have only one role, Approver. Updated the VPP and exported a new copy of the layout as v7.
- **Yash:** Added Actions to Events, a Reactors list of the classes that should react to an event, and a template method for the different action types.
- **Yash:** Made the final change to observers and subjects. `update` is no longer called on observers directly. The action observer handles it in a templatized function with the same steps every time, and each observer type overrides only the step it needs.

Major changes we made

- Moved from State to Visitor as advised by prof marshal
- Created Templates to create custom handlers and strategies
- Created Custom Events and an event registry to manage those events

Task8:

Github: https://github.com/yashGhela/cos214prac6.git